

## 1. UN LENGUAJE IMPERATIVO SIMPLE

**Primera aproximación.** Vamos a introducir los conceptos centrales desde un fragmento (llamado LIS), que refleja las construcciones más significativas de este tipo de lenguajes.

La gramática abstracta que define LIS es la siguiente:

$$\begin{aligned}
\langle comm \rangle &::= \text{skip} \\
&\quad \langle var \rangle := \langle interp \rangle \\
&\quad \langle comm \rangle ; \langle comm \rangle \\
&\quad \text{if } \langle boolexp \rangle \text{ then } \langle comm \rangle \text{ else } \langle comm \rangle \\
&\quad \text{newvar } \langle var \rangle := \langle interp \rangle \text{ in } \langle comm \rangle \\
&\quad \text{while } \langle boolexp \rangle \text{ do } \langle comm \rangle \\
\\
\langle interp \rangle &::= \langle natconst \rangle \quad \langle boolexp \rangle ::= \langle boolconst \rangle \\
&\quad \langle var \rangle \quad \langle interp \rangle = \langle interp \rangle \\
&\quad \langle interp \rangle + \langle interp \rangle \quad \langle interp \rangle < \langle interp \rangle \\
&\quad \langle interp \rangle * \langle interp \rangle \quad \langle interp \rangle \leq \langle interp \rangle \\
&\quad \langle interp \rangle - \langle interp \rangle \quad \langle interp \rangle > \langle interp \rangle \\
&\quad \langle interp \rangle / \langle interp \rangle \quad \langle interp \rangle \geq \langle interp \rangle \\
&\quad \langle interp \rangle \% \langle interp \rangle \quad \neg \langle boolexp \rangle \\
&\quad - \langle interp \rangle \quad \langle boolexp \rangle \vee \langle boolexp \rangle \\
&\quad \quad \langle boolexp \rangle \wedge \langle boolexp \rangle \\
\\
\langle natconst \rangle &::= 0 \mid 1 \mid 2 \mid \dots \\
\langle boolconst \rangle &::= \text{true} \mid \text{false}
\end{aligned}$$

Note que las expresiones enteras son las mismas que en la lógica de predicados, y las expresiones booleanas también salvo que ahora, como es habitual en los lenguajes de programación, no hay cuantificadores, y por eso usamos un nombre diferente:  $\langle boolexp \rangle$  en vez de  $\langle assert \rangle$ .

El lenguaje LIS, en su búsqueda de simpleza posee varios aspectos reduccionistas. El primero que podemos mencionar (además de haber sólo dos tipos de expresiones: enteras y booleanas) es que las expresiones enteras siempre se pueden evaluar, y su resultado es un entero. Al igual que en el Cálculo de Predicados, seguimos asumiendo que todas las funciones primitivas (incluida la división) son funciones totales.

Para comenzar, excluimos de nuestra consideración al comando de iteración **while**. Veremos ahora como daríamos significado al lenguaje sin iteración, es decir un lenguaje en el cual todos los programas terminan (ni se cuelgan ni hay errores de operación).

LIS sólo posee variables que adoptan valores enteros. Luego la noción de estado se refleja en la siguiente definición:

**Conjunto de estados:**  $\Sigma = \langle var \rangle \rightarrow \mathbf{Z}$

Esto es, el conjunto de estados  $\Sigma$  no es otra cosa que la familia de funciones totales entre el conjunto de variables y los enteros. Aquí se refleja otra característica reduccionista de LIS: la memoria posee infinitos lugares que siempre alojan un número entero.

El significado de los comandos y las expresiones del lenguaje LIS sin iteración estará dado por las siguientes funciones semánticas:

$$\begin{aligned}
\llbracket \_ \rrbracket^{interp} &\in \langle interp \rangle \rightarrow \Sigma \rightarrow \mathbf{Z} \\
\llbracket \_ \rrbracket^{boolexp} &\in \langle boolexp \rangle \rightarrow \Sigma \rightarrow \{V, F\} \\
\llbracket \_ \rrbracket^{comm} &\in \langle comm \rangle \rightarrow \Sigma \rightarrow \Sigma
\end{aligned}$$

Debemos notar que dado que no estamos considerando la iteración en LIS, todos los comandos terminan en su ejecución, y su “resultado” es un nuevo estado.

Las ecuaciones semánticas para las frases de tipo  $\langle interp \rangle$  y  $\langle boolexp \rangle$  son similares a las dadas para el lenguaje del Cálculo de Predicados. Las ecuaciones semánticas para las frases de tipo  $\langle comm \rangle$  son las siguientes. Las primeras cuatro no merecen mayores comentarios:

$$\begin{aligned}
\llbracket \text{skip} \rrbracket \sigma &= \sigma \\
\llbracket v := e \rrbracket \sigma &= [\sigma|v : \llbracket e \rrbracket \sigma] \\
\llbracket c_0; c_1 \rrbracket \sigma &= \llbracket c_1 \rrbracket (\llbracket c_0 \rrbracket \sigma) \\
\llbracket \text{if } e \text{ then } c \text{ else } c' \rrbracket \sigma &= \text{if } \llbracket e \rrbracket \sigma \text{ then } \llbracket c \rrbracket \sigma \text{ else } \llbracket c' \rrbracket \sigma
\end{aligned}$$

Por ejemplo:

$$\begin{aligned}
\llbracket x := x - 1 \rrbracket \sigma &= [\sigma|x : \llbracket x - 1 \rrbracket \sigma] \\
&= [\sigma|x : \llbracket x \rrbracket \sigma - \llbracket 1 \rrbracket \sigma] \\
&= [\sigma|x : \sigma x - 1] \\
\llbracket y := y + x \rrbracket \sigma &= [\sigma|y : \llbracket y + x \rrbracket \sigma] \\
&= [\sigma|y : \llbracket y \rrbracket \sigma + \llbracket x \rrbracket \sigma] \\
&= [\sigma|y : \sigma y + \sigma x]
\end{aligned}$$

Para definir el significado de las variables locales debemos expresar el hecho de que el valor original del estado en la variable local se restaura al terminar la ejecución de su alcance. (Definimos a  $c$  como el alcance de **newvar**  $v := e$  in  $c$ .) Este hecho lo expresamos mediante la aplicación de la función “restauración de  $v$  según  $\sigma$ ”:

$$f_{v,\sigma}\sigma' = [\sigma'|v : \sigma v]$$

De esta manera podemos expresar el significado de la construcción **newvar**:

$$\llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma = f_{v,\sigma}(\llbracket c \rrbracket [\sigma|v : \llbracket e \rrbracket \sigma])$$

Dado que estas funciones auxiliares serán frecuentemente utilizadas, es conveniente incorporar un mecanismo que nos permita expresar su efecto sin necesidad de introducir nuevos símbolos de función (en este caso  $f_{v,\sigma}$ ). Este mecanismo es la notación lambda, que

por ahora será para nosotros una herramienta del metalenguaje apropiada para describir funciones (ver descripción abajo). La función "restauración" adopta la siguiente forma:

$$f_{v,\sigma} = \lambda\sigma' \in \Sigma. [\sigma'|v : \sigma v]$$

De esta manera podemos expresar en el metalenguaje esta operación sin necesidad de introducir funciones auxiliares:

$$\llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma = (\lambda\sigma' \in \Sigma. [\sigma'|v : \sigma v]) (\llbracket c \rrbracket [\sigma|v : \llbracket e \rrbracket \sigma])$$

**La notación lambda.** Explicaremos ahora brevemente el uso de esta notación. Podemos decir que el resultado de la función entera  $f(x) = 2x + 5$  "aplicada" a 3 es  $2 * 3 + 5$ , por que el mismo se obtiene producto de la sustitución en la expresión  $2x + 5$  de  $x$  por 3. La notación lambda nos provee de una forma adecuada de definir funciones de manera que la operatoria con las mismas que involucran "aplicaciones" de funciones a otras expresiones se efectúe apropiadamente. La misma consiste en señalar en una expresión que variable "determina" la función, producto precisamente, de su variación. Mediante la expresión

$$\lambda x. 2x + 5$$

denotamos la función  $f$  (sin necesidad de introducir un nuevo símbolo). Podemos operar por ejemplo de la siguiente manera:

$$(\lambda x. 2x + 5)(4z) = 8z + 5$$

Note que la expresión  $\lambda x. 2x + 5$  tiene un significado que no depende de  $x$ , de hecho podríamos escribir equivalentemente  $\lambda w. 2w + 5$ .

**1.1. Dificultades para dar significado a la iteración.** En los lenguajes imperativos, la estructura típica de recursión es la iteración, que tiene la forma del comando:

**while  $b$  do  $c$**

El significado operacional usual establece que se pregunta si  $b$  es verdadero, y en ese caso se ejecuta  $c$ , y luego se continua en el estado corriente ejecutando nuevamente el while. En nuestro marco de significado, escribiríamos este hecho de la siguiente manera:

$$\begin{aligned} \llbracket \text{while } b \text{ do } c \rrbracket \sigma &= \text{if } \llbracket b \rrbracket \sigma \text{ then } \llbracket c; \text{while } b \text{ do } c \rrbracket \sigma \text{ else } \llbracket \text{skip} \rrbracket \sigma \\ &= \begin{cases} \llbracket \text{while } b \text{ do } c \rrbracket (\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \\ \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \end{cases} \end{aligned}$$

Surgen varias dificultades al pretender tomar esto como una definición. La primera dificultad es que, como en el caso de las funciones recursivas, el dominio semántico es inadecuado. Al incluir la iteración se incorpora otra situación de ejecución atípica: la no terminación del programa.

Incorporaremos entonces el símbolo de "no terminación"  $\perp$  al dominio de resultados posibles. De manera que nuestro nuevo "espacio de significado" será

$$\Sigma_{\perp} = \Sigma \cup \{\perp\}$$

$\Sigma_{\perp}$  representa el conjunto de resultados posibles de la ejecución de un programa imperativo. El mismo cuenta por supuesto con una copia de  $\Sigma$ , representando a los resultados de programas que terminan en un estado. Además de la no terminación, en el futuro incorporará la posibilidad de representar otros comportamientos, como los errores de ejecución, y los inputs y outputs, por ejemplo.

Si volvemos ahora a la definición de arriba, la expresión

$$\llbracket \text{while } b \text{ do } c \rrbracket (\llbracket c \rrbracket \sigma) \quad (2)$$

tiene el problema de que  $\llbracket c \rrbracket \sigma$  puede ser  $\perp$ , que no pertenece al dominio de  $\llbracket \text{while } b \text{ do } c \rrbracket$ . Esto se soluciona acudiendo a nuestras funciones auxiliares de transferencia de control. Si  $f \in \Sigma \rightarrow \Sigma_{\perp}$ , entonces definimos una nueva función  $f_{\perp} \in \Sigma_{\perp} \rightarrow \Sigma_{\perp}$  de la siguiente manera:

$$f_{\perp} \sigma = \sigma \quad f_{\perp} \perp = \perp$$

Luego la expresión (2) puede ser corregida poniendo

$$\llbracket \text{while } b \text{ do } c \rrbracket_{\perp} (\llbracket c \rrbracket \sigma)$$

Pero el problema principal de querer tomar (1) como definición es la no dirección por sintaxis: la definición de  $\llbracket \text{while } b \text{ do } c \rrbracket$  no se da exclusivamente en términos de  $\llbracket b \rrbracket$  y  $\llbracket c \rrbracket$ . Más aún, la definición es circular: se usa  $\llbracket \text{while } b \text{ do } c \rrbracket$  para definir  $\llbracket \text{while } b \text{ do } c \rrbracket$ , cayendo en un confuso estilo de definición autoreferente, develando así la naturaleza recursiva del while, y con ella los problemas que estuvimos viendo para las funciones recursivas enteras.

El análisis hecho para las funciones recursivas enteras de Haskell es útil para entender ahora el problema de la iteración:

- el significado de  $\llbracket \text{while } b \text{ do } c \rrbracket$  es una función  $\omega \in \Sigma \rightarrow \Sigma_{\perp}$
- tal función  $\omega$  satisface la siguiente "ecuación funcional":

$$\omega \sigma = \begin{cases} \omega_{\perp} (\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \\ \sigma & \text{si } \neg \llbracket b \rrbracket \sigma \end{cases} \quad (W)$$

Obtenemos entonces una ecuación similar a la ecuación (ER). Nuevamente, describir el conjunto de soluciones de la ecuación (W) de manera genérica es matemáticamente complicado ya que puede haber infinitas funciones. Pero al igual que para las funciones recursivas nos interesa la solución "menos informativa" (o sea la menor). El Teorema del Menor Punto Fijo nos dará entonces la respuesta.

Un caso extremo de solución "menos informativa" lo constituye el significado de

**while true do skip,**

que es claramente la función idénticamente bottom:

$$\llbracket \text{while true do skip} \rrbracket = \omega \quad \omega \sigma = \perp \quad (\text{para todo } \sigma \in \Sigma)$$

$$\begin{aligned}\omega\sigma &= \begin{cases} \omega_{\perp}(\llbracket\text{skip}\rrbracket\sigma) & \text{si } \llbracket\text{true}\rrbracket\sigma \\ \sigma & \text{si } \neg\llbracket\text{true}\rrbracket\sigma \end{cases} \\ &= \omega_{\perp}\sigma \\ &= \omega\sigma\end{aligned}$$

- (1) ¿Cuál es el significado de **while false do c**? Debe dar una función  $\omega$  perteneciente a  $\Sigma \rightarrow \Sigma_{\perp}$ .
- (2) ¿Cuál es el significado de **while  $x < 0$  do skip**? Debe dar una función  $\omega$  perteneciente a  $\Sigma \rightarrow \Sigma_{\perp}$ .
- (3) Según lo visto hasta el momento, ¿qué puede decir sobre el significado de **while b do c**?

$$\begin{array}{cccccccc} \dots & \sigma & \sigma' & \sigma'' & \sigma_0 & \sigma_1 & \sigma_2 & \sigma_3 & \dots \\ & & & \backslash & | & / & & & \\ & & & & \perp & & & & \end{array}$$
$$\llbracket c_0; c_1 \rrbracket \sigma = \begin{cases} \perp & \text{si } \llbracket c_0 \rrbracket \sigma = \perp \\ \llbracket c_1 \rrbracket (\llbracket c_0 \rrbracket \sigma) & \text{si no} \end{cases}$$
$$\begin{aligned}
[x := x - 1; y := y + x] \sigma &= [y := y + x] \perp ([x := x - 1] \sigma) \\
&= [y := y + x] \perp [\sigma | x : \sigma \ x - 1] \\
&= [y := y + x] [\sigma | x : \sigma \ x - 1] \\
&= [y := y + x] [\overbrace{\sigma | x : \sigma \ x - 1}^{\sigma'}] \\
&= [y := y + x] \sigma' \\
&= [\sigma' | y : \sigma' \ y + \sigma' \ x] \\
&= [\sigma | x : \sigma \ x - 1 | y : \sigma \ y + \sigma \ x - 1]
\end{aligned}$$
$$\begin{aligned} \llbracket \text{while } b \text{ do } c \rrbracket \sigma &= \llbracket \text{if } b \text{ then } c; \text{ while } b \text{ do } c \text{ else skip} \rrbracket \sigma \\ &= \begin{cases} \llbracket \text{while } b \text{ do } c \rrbracket_{\perp} (\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \\ \sigma & \text{si no} \end{cases} \end{aligned}$$

resulta una ecuación que no es dirigida por sintaxis, pero que por tener el formato de una ecuación recursiva podemos recurrir al Teorema del Menor Punto Fijo para caracterizar la solución que buscamos. Definamos

$$F w \sigma = \begin{cases} w_{\perp}(\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \\ \sigma & \text{si no} \end{cases}$$

con esta definición, la ecuación del **while** queda

$$\llbracket \text{while } b \text{ do } c \rrbracket \sigma = F \llbracket \text{while } b \text{ do } c \rrbracket \sigma$$

Note que estamos planteando una igualdad de funciones. Es fácil comprobar que

$$\llbracket \text{while } b \text{ do } c \rrbracket \in \Sigma \rightarrow \Sigma_{\perp} \text{ y } F \in (\Sigma \rightarrow \Sigma_{\perp}) \rightarrow (\Sigma \rightarrow \Sigma_{\perp})$$

Por ello,  $F \llbracket \text{while } b \text{ do } c \rrbracket \in \Sigma \rightarrow \Sigma_{\perp}$ , luego la ecuación también puede escribirse

$$\llbracket \text{while } b \text{ do } c \rrbracket = F \llbracket \text{while } b \text{ do } c \rrbracket$$

que simplemente dice que  $\llbracket \text{while } b \text{ do } c \rrbracket$  es punto fijo de  $F$ . Recordemos entonces el teorema del menor punto fijo.

*Teorema.* Sea  $D$  un dominio, y  $F \in D \rightarrow D$  continua. Entonces  $\sup(F^i \perp)$  existe y es el menor punto fijo de  $F$ .

Recordemos que nos interesa el menor punto fijo de  $F$  porque otros puntos fijos contienen información que no se encuentra en la ecuación a resolver.

Sabemos que  $F \in (\Sigma \rightarrow \Sigma_{\perp}) \rightarrow (\Sigma \rightarrow \Sigma_{\perp})$ , es decir  $F \in D \rightarrow D$  con  $D = \Sigma \rightarrow \Sigma_{\perp}$ , que es un dominio porque  $\Sigma_{\perp}$  lo es.

Para poder utilizar el teorema, deberíamos asegurarnos de que  $F$  es continua. En caso de serlo, la semántica de **while**  $b$  **do**  $c$  será

$$\llbracket \text{while } b \text{ do } c \rrbracket = \bigsqcup_{i=0}^{\infty} F^i \perp_{\Sigma \rightarrow \Sigma_{\perp}}$$

para

$$F w \sigma = \begin{cases} w_{\perp}(\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \\ \sigma & \text{si no} \end{cases}$$

Esta definición sí es dirigida por sintaxis.

Veamos que  $F$  es continua. Para ello, primero vemos que es monótona: sean  $w \sqsubseteq w' \in \Sigma \rightarrow \Sigma_{\perp}$ . Si  $\llbracket b \rrbracket \sigma$  y  $\llbracket c \rrbracket \sigma = \perp$ , entonces

$$\begin{aligned} F w \sigma &= w_{\perp}(\llbracket c \rrbracket \sigma) && \text{definición de } F \\ &= \perp && \llbracket c \rrbracket \sigma = \perp \\ &\leq F w' \sigma && \perp \text{ es el mínimo} \end{aligned}$$

En cambio, si  $\llbracket b \rrbracket \sigma$  pero  $\llbracket c \rrbracket \sigma \neq \perp$

$$\begin{aligned} F w \sigma &= w_{\perp}(\llbracket c \rrbracket \sigma) && \text{definición de } F \\ &= w(\llbracket c \rrbracket \sigma) && \llbracket c \rrbracket \sigma \neq \perp \\ &\leq w'(\llbracket c \rrbracket \sigma) && w \sqsubseteq w' \\ &= w'_{\perp}(\llbracket c \rrbracket \sigma) && \llbracket c \rrbracket \sigma \neq \perp \\ &= F w' \sigma && \text{definición de } F \end{aligned}$$

Por último, si  $\neg \llbracket b \rrbracket \sigma$

$$\begin{aligned} F w \sigma &= \sigma \\ &= F w' \sigma \end{aligned}$$

Por lo tanto,  $F$  es monótona. Por las propiedades ya demostradas, para comprobar continuidad, alcanza con demostrar que para toda cadena  $w_0 \sqsubseteq w_1 \sqsubseteq w_2 \sqsubseteq \dots$  de funciones de  $\Sigma \rightarrow \Sigma_{\perp}$ ,  $F(\bigsqcup_{i=0}^{\infty} w_i) \sqsubseteq \bigsqcup_{i=0}^{\infty} F w_i$ .

Nuevamente consideramos tres casos. Si  $\llbracket b \rrbracket \sigma$  y  $\llbracket c \rrbracket \sigma = \perp$ , entonces

$$\begin{aligned} F(\bigsqcup_{i=0}^{\infty} w_i) \sigma &= (\bigsqcup_{i=0}^{\infty} w_i)_{\perp}(\llbracket c \rrbracket \sigma) && \text{definición de } F \\ &= \perp && \llbracket c \rrbracket \sigma = \perp \\ &\leq \bigsqcup_{i=0}^{\infty} F w_i \sigma && \perp \text{ es el mínimo} \end{aligned}$$

Ahora, si  $\llbracket b \rrbracket \sigma$  pero  $\llbracket c \rrbracket \sigma \neq \perp$

$$\begin{aligned} F(\bigsqcup_{i=0}^{\infty} w_i) \sigma &= (\bigsqcup_{i=0}^{\infty} w_i)_{\perp}(\llbracket c \rrbracket \sigma) && \text{definición de } F \\ &= (\bigsqcup_{i=0}^{\infty} w_i)(\llbracket c \rrbracket \sigma) && \llbracket c \rrbracket \sigma \neq \perp \\ &= \bigsqcup_{i=0}^{\infty} w_i(\llbracket c \rrbracket \sigma) && \text{supremo de cadena de funciones} \\ &= \bigsqcup_{i=0}^{\infty} w_{i\perp}(\llbracket c \rrbracket \sigma) && \llbracket c \rrbracket \sigma \neq \perp \\ &= \bigsqcup_{i=0}^{\infty} F w_i \sigma && \text{definición de } F \\ &= (\bigsqcup_{i=0}^{\infty} F w_i) \sigma && \text{supremo de cadena de funciones} \end{aligned}$$

Finalmente, si  $\neg \llbracket b \rrbracket \sigma$ ,

$$\begin{aligned} F(\bigsqcup_{i=0}^{\infty} w_i) \sigma &= \sigma && \text{definición de } F \\ &= \bigsqcup_{i=0}^{\infty} \sigma && \text{supremo de cadena constante} \\ &= \bigsqcup_{i=0}^{\infty} F w_i \sigma && \text{definición de } F \\ &= (\bigsqcup_{i=0}^{\infty} F w_i) \sigma && \text{supremo de cadena de funciones} \end{aligned}$$

Por lo tanto  $F$  es continua y la definición dada para la semántica del **while** está bien definida.

**1.3. Ejemplo.** Podemos considerar algunos ejemplos de cálculo de semántica de programas con **while**. Revisemos primero el programa **while true do skip** que es el caso más sencillo de un programa que no termina en ningún estado, y por ello, su semántica debe ser  $\perp_{\Sigma \rightarrow \Sigma_{\perp}}$ .



Sea  $F \in (\Sigma \rightarrow \Sigma_{\perp}) \rightarrow (\Sigma \rightarrow \Sigma_{\perp})$  definida por

$$\begin{aligned} F w \sigma &= \begin{cases} w_{\perp}(\llbracket \text{skip} \rrbracket \sigma) & \text{si } \llbracket \text{true} \rrbracket \sigma \\ \sigma & \text{si no} \end{cases} \\ &= w_{\perp}(\llbracket \text{skip} \rrbracket \sigma) \\ &= w_{\perp} \sigma \\ &= w \sigma \end{aligned}$$

es decir,  $F w = w$ . Calculemos  $F^i \perp_{\Sigma \rightarrow \Sigma_{\perp}}$  para algunos  $i \in \mathbb{N}$ :

$$\begin{aligned} F^0 \perp_{\Sigma \rightarrow \Sigma_{\perp}} &= \perp_{\Sigma \rightarrow \Sigma_{\perp}} \\ F^1 \perp_{\Sigma \rightarrow \Sigma_{\perp}} &= F(F^0 \perp_{\Sigma \rightarrow \Sigma_{\perp}}) \\ &= F \perp_{\Sigma \rightarrow \Sigma_{\perp}} \\ &= \perp_{\Sigma \rightarrow \Sigma_{\perp}} \end{aligned}$$

Tenemos entonces  $F^1 \perp_{\Sigma \rightarrow \Sigma_{\perp}} = F^0 \perp_{\Sigma \rightarrow \Sigma_{\perp}}$ . En general, si tenemos  $F^{i+1} \perp_D = F^i \perp_D$  podemos concluir que  $F^i \perp_D = F^{i+1} \perp_D = F^{i+2} \perp_D = \dots$  y la cadena resulta no interesante y su supremo es  $F^i \perp_D$ .

En el caso de **while true do skip** obtenemos,  $\bigsqcup_{i=0}^{\infty} F^i \perp_{\Sigma \rightarrow \Sigma_{\perp}} = \perp_{\Sigma \rightarrow \Sigma_{\perp}}$  que es lo que habíamos intuido.

Podíamos haber observado también, al obtener  $F w = w$ , que cualquier  $w \in \Sigma \rightarrow \Sigma_{\perp}$  es punto fijo de  $F$  ya que  $F$  es la función identidad. El menor de ellos es el mínimo de  $\Sigma \rightarrow \Sigma_{\perp}$ , o sea,  $\perp_{\Sigma \rightarrow \Sigma_{\perp}}$ .

Un ejemplo no trivial es el de **while**  $x \neq 0 \wedge x \neq 1$  **do**  $x := x - 2$ . Intuitivamente, este programa reemplaza el valor de  $x$  en el estado por el del módulo 2 de dicho valor si el mismo es no negativo. Si es negativo, no termina. Comparar con el caso considerado anteriormente de una función recursiva que tenía a *mod2* como solución.

Tenemos

$$\llbracket \text{while } x \neq 0 \wedge x \neq 1 \text{ do } x := x - 2 \rrbracket = \bigsqcup_{i=0}^{\infty} F^i \perp_{\Sigma \rightarrow \Sigma_{\perp}}$$

para

$$\begin{aligned} F w \sigma &= \begin{cases} w_{\perp}(\llbracket x := x - 2 \rrbracket \sigma) & \text{si } \llbracket x \neq 0 \wedge x \neq 1 \rrbracket \sigma \\ \sigma & \text{si no} \end{cases} \\ &= \begin{cases} w_{\perp}(\llbracket \sigma | x : \sigma x - 2 \rrbracket) & \text{si } \sigma x \notin \{0, 1\} \\ \sigma & \text{si no} \end{cases} \\ &= \begin{cases} w(\llbracket \sigma | x : \sigma x - 2 \rrbracket) & \text{si } \sigma x \notin \{0, 1\} \\ \sigma & \text{si no} \end{cases} \\ &= \begin{cases} \sigma & \text{si } \sigma x \in \{0, 1\} \\ w[\sigma | x : \sigma x - 2] & \text{si no} \end{cases} \end{aligned}$$

Calculemos  $F^i \perp_{\Sigma \rightarrow \Sigma_{\perp}}$  para algunos  $i \in \mathbb{N}$ :

$$\begin{aligned} F^0 \perp_{\Sigma \rightarrow \Sigma_{\perp}} &= \perp_{\Sigma \rightarrow \Sigma_{\perp}} \\ F^1 \perp_{\Sigma \rightarrow \Sigma_{\perp}} &= F(F^0 \perp_{\Sigma \rightarrow \Sigma_{\perp}}) \\ &= F \perp_{\Sigma \rightarrow \Sigma_{\perp}} \\ &= \sigma \mapsto \begin{cases} \sigma & \text{si } \sigma x \in \{0, 1\} \\ \perp_{\Sigma \rightarrow \Sigma_{\perp}}[\sigma | x : \sigma x - 2] & \text{si no} \end{cases} \\ &= \sigma \mapsto \begin{cases} \sigma & \text{si } \sigma x \in \{0, 1\} \\ \perp & \text{si no} \end{cases} \\ F^2 \perp_{\Sigma \rightarrow \Sigma_{\perp}} &= F(F^1 \perp_{\Sigma \rightarrow \Sigma_{\perp}}) \\ &= \sigma \mapsto \begin{cases} \sigma & \text{si } \sigma x \in \{0, 1\} \\ F^1 \perp_{\Sigma \rightarrow \Sigma_{\perp}}[\sigma | x : \sigma x - 2] & \text{si no} \\ \sigma & \text{si } \sigma x \in \{0, 1\} \end{cases} \\ &= \sigma \mapsto \begin{cases} F^1 \perp_{\Sigma \rightarrow \Sigma_{\perp}}[\overbrace{\sigma | x : \sigma x - 2}^{\sigma'}] & \text{si no} \\ \sigma & \text{si } \sigma x \in \{0, 1\} \end{cases} \\ &= \sigma \mapsto \begin{cases} \sigma' & \text{si } \sigma x \notin \{0, 1\} \wedge \sigma' x \in \{0, 1\} \\ \perp & \text{si } \sigma x \notin \{0, 1\} \wedge \sigma' x \notin \{0, 1\} \end{cases} \\ &= \sigma \mapsto \begin{cases} \sigma & \text{si } \sigma x \in \{0, 1\} \\ \llbracket \sigma | x : \sigma x - 2 \rrbracket & \text{si } \sigma x \notin \{2, 3\} \\ \perp & \text{si } \sigma x \notin \{0, 1, 2, 3\} \end{cases} \\ &= \sigma \mapsto \begin{cases} \llbracket \sigma | x : \sigma x \% 2 \rrbracket & \text{si } \sigma x \notin \{0, 1, 2, 3\} \\ \perp & \text{si } \sigma x \notin \{0, 1, 2, 3\} \end{cases} \end{aligned}$$

$$\begin{aligned} F^3 \perp_{\Sigma \rightarrow \Sigma_{\perp}} &= F(F^2 \perp_{\Sigma \rightarrow \Sigma_{\perp}}) \\ &= \sigma \mapsto \begin{cases} \sigma & \text{si } \sigma x \in \{0, 1\} \\ F^2 \perp_{\Sigma \rightarrow \Sigma_{\perp}}[\overbrace{\sigma | x : \sigma x - 2}^{\sigma'}] & \text{si no} \end{cases} \\ &= \sigma \mapsto \begin{cases} \sigma & \text{si } \sigma x \in \{0, 1\} \\ \llbracket \sigma' | x : \sigma' x \% 2 \rrbracket & \text{si } \sigma x \notin \{0, 1\} \wedge \sigma' x \in \{0, 1, 2, 3\} \\ \perp & \text{si } \sigma x \notin \{0, 1\} \wedge \sigma' x \notin \{0, 1, 2, 3\} \end{cases} \\ &= \sigma \mapsto \begin{cases} \sigma & \text{si } \sigma x \in \{0, 1\} \\ \llbracket \sigma' | x : \sigma' x \% 2 \rrbracket & \text{si } \sigma x \in \{2, 3, 4, 5\} \\ \perp & \text{si } \sigma x \notin \{0, 1, 2, 3, 4, 5\} \end{cases} \\ &= \sigma \mapsto \begin{cases} \llbracket \sigma | x : \sigma x \% 2 \rrbracket & \text{si } \sigma x \in \{0, 1, 2, 3, 4, 5\} \\ \perp & \text{si } \sigma x \notin \{0, 1, 2, 3, 4, 5\} \end{cases} \end{aligned}$$

Se puede comprobar que

$$F^i \perp_{\Sigma \rightarrow \Sigma_{\perp}} = \sigma \mapsto \begin{cases} \llbracket \sigma | x : \sigma x \% 2 \rrbracket & \text{si } \sigma x \in \{0, 1, \dots, 2 * i - 1\} \\ \perp & \text{si } \sigma x \notin \{0, 1, \dots, 2 * i - 1\} \end{cases}$$

Observar que  $F^i \perp_{\Sigma \rightarrow \Sigma_{\perp}}$  es la función que, aplicada a un estado para el que alcanzarían con (a lo sumo)  $i - 1$  iteraciones del ciclo para terminar, devuelve el estado final del **while**,

y aplicada a un estado para el que no alcanzarían, devuelve  $\perp$ . En el límite esto daría la función que aplicada a un estado para el que alcanza con una cantidad finita de iteraciones, devuelve el estado final, y aplicada a un estado para el que no termina, devuelve  $\perp$ :

$$\bigcap_{i=0}^{\infty} F^i \perp_{\Sigma \rightarrow \Sigma} = \sigma \mapsto \begin{cases} [\sigma|x : \sigma \text{ x}\%2] & \text{si } \sigma \text{ x} \geq 0 \\ \perp & \text{si no} \end{cases}$$

que es lo que habíamos anticipado originariamente.

## 2. PROPIEDADES DE LA SEMÁNTICA

**Variables Libres, Variables asignables.** Similar a los predicados, el conjunto de variables libres de un comando se define con las siguientes ecuaciones.

$$\begin{aligned} FV(\text{skip}) &= \emptyset \\ FV(v := e) &= \{v\} \cup FV(e) \\ FV(c_0; c_1) &= FV(c_0) \cup FV(c_1) \\ FV(\text{if } b \text{ then } c_0 \text{ else } c_1) &= FV(b) \cup FV(c_0) \cup FV(c_1) \\ FV(\text{while } b \text{ do } c) &= FV(b) \cup FV(c) \\ FV(\text{newvar } v := e \text{ in } c) &= FV(e) \cup (FV(c) - \{v\}) \end{aligned}$$

El conjunto de las variables que eventualmente modifican su estado en la ejecución de un comando se define con las siguientes ecuaciones.

$$\begin{aligned} FA(\text{skip}) &= \emptyset \\ FA(v := e) &= \{v\} \\ FA(c_0; c_1) &= FA(c_0) \cup FA(c_1) \\ FA(\text{if } b \text{ then } c_0 \text{ else } c_1) &= FA(c_0) \cup FA(c_1) \\ FA(\text{while } b \text{ do } c) &= FA(c) \\ FA(\text{newvar } v := e \text{ in } c) &= FV(c) - \{v\} \end{aligned}$$

Claramente  $FA(c) \subseteq FV(c)$ , como puede comprobarse ecuación por ecuación.

**Teorema de Coincidencia (TC).** Si dos estados  $\sigma$  y  $\sigma'$  coinciden en las variables libres de  $c$ , entonces da lo mismo evaluar  $c$  en  $\sigma$  o  $\sigma'$ .

Note que debemos tener cuidado en como esta proposición se escribe, ya que si enunciáramos el hecho de manera similar a los predicados tenemos una proposición falsa:

$$(\forall w \in FV(c). \sigma w = \sigma' w) \Rightarrow \llbracket c \rrbracket \sigma = \llbracket c \rrbracket \sigma'$$

Esto no vale si  $c$  es un comando, ya que devuelven estados finales que pueden diferir en las  $w \notin FV(c)$  (porque tales diferencias podían existir previamente). Además, podría ocurrir que uno de ellos no termine, en cuyo caso ni siquiera devolvería un estado. Puede verse que si uno de ellos se cuelga, el otro también.

El enunciado correcto del **Teorema de Coincidencia** es:

- (1)  $(\forall w \in FV(c). \sigma w = \sigma' w)$  implica,
  - o bien  $\llbracket c \rrbracket \sigma = \perp = \llbracket c \rrbracket \sigma'$ ,
  - o bien  $\llbracket c \rrbracket \sigma \neq \perp \neq \llbracket c \rrbracket \sigma'$  y  $\forall w \in FV(c). \llbracket c \rrbracket \sigma w = \llbracket c \rrbracket \sigma' w$

- (2) Si  $\llbracket c \rrbracket \sigma \neq \perp$ , entonces  $\forall w \notin FA(c). \llbracket c \rrbracket \sigma w = \sigma w$ .

**Teorema de Renombre (TR).** Otra propiedad que cobra importancia en el lenguaje imperativo es la que establece que no importa el nombre de las variables utilizadas en las declaraciones de variables locales (o sea las ligadas):

$$u \notin FV(c) - \{v\} \Rightarrow \llbracket \text{newvar } u := e \text{ in } c/v \rightarrow u \rrbracket = \llbracket \text{newvar } v := e \text{ in } c \rrbracket$$

La demostración del TR surge inmediatamente del Teorema de Sustitución, que ahora abordaremos.

**Sustituciones.** Por culpa de la asignación  $v := e$ , no podemos reemplazar una variable por una expresión entera arbitraria en un comando. Sólo podemos reemplazarla por otra variable. Luego consideraremos como el conjunto de las sustituciones al siguiente

$$\Delta = \langle var \rangle \rightarrow \langle var \rangle$$

A pesar de este cambio, las ecuaciones que la definían para la lógica de predicados siguen valiendo (salvo el caso de los cuantificadores, que ahora no tenemos). Ahora se agregan ecuaciones para los comandos.

$$\_ / \_ \in \langle comm \rangle \times \Delta \rightarrow \langle comm \rangle$$

$$\begin{aligned} \text{skip} / \delta &= \text{skip} \\ (v := e) / \delta &= (\delta v) := (e / \delta) \\ (c_0; c_1) / \delta &= (c_0 / \delta); (c_1 / \delta) \\ (\text{if } b \text{ then } c_0 \text{ else } c_1) / \delta &= \text{if } b / \delta \text{ then } c_0 / \delta \text{ else } c_1 / \delta \\ (\text{while } b \text{ do } c) / \delta &= \text{while } b / \delta \text{ do } c / \delta \end{aligned}$$

El caso del newvar tiene los mismos inconvenientes que los cuantificadores, ahora es un poco más simple ya que  $\delta w$  son siempre variables.

$$(\text{newvar } v := e \text{ in } c) / \delta = \text{newvar } v_{new} := e / \delta \text{ in } (c / [\delta | v : v_{new}])$$

donde  $v_{new} \notin \{\delta w | w \in FV(c) - \{v\}\}$

El enunciado del Teorema de sustitución se complica bastante en relación al correspondiente Teorema del cálculo de predicados. La cuestión es: ¿si aplico la sustitución  $\delta$  a  $c$  y luego evalúo en el estado  $\sigma$ , puedo obtener el mismo resultado a partir de  $c$  sin sustituir si evalúo en un estado que hace el trabajo de  $\delta$  y de  $\sigma$  (en las variables libres de  $c$ )?

O sea, vale

$$(\forall w \in FV(c). \sigma(\delta w) = \sigma' w) \Rightarrow \llbracket c / \delta \rrbracket \sigma = \llbracket c \rrbracket \sigma'?$$

Por los problemas mencionados para el TC, esta propiedad debería reescribirse atentos a la posibilidad de que los estados originales adopten valores distintos en las variables que no ocurren libres en  $c$ , y en consecuencia los estados finales difieran en las mismas. Deberíamos comparar entonces  $\llbracket c / \delta \rrbracket \sigma(\delta w)$  con  $\llbracket c \rrbracket \sigma' w$ . Por ejemplo, si al programa  $c = (x := x - 1; y := 2 * y)$  se le sustituye  $x$  por  $u$  e  $y$  por  $v$  (sustitución  $\delta$ ), entonces al ejecutar

$c/\delta$  en un estado  $\sigma$  deberíamos obtener en  $u$  y  $v$  los valores que se obtienen al ejecutar  $c$  en un estado en donde  $x$  e  $y$  adopten los valores de  $\sigma u$  y  $\sigma v$  resp.

Pero ni siquiera así vale el TS. El problema ocurre cuando  $\delta$  manda dos variables a una misma variable. Por ejemplo, consideremos el programa  $x := x - 1; y := 2 * y$ . Si tenemos la mala suerte de que  $\delta$  reemplace  $x$  e  $y$  por la misma variable  $z$ , entonces el programa resultante será  $z := z - 1; z := 2 * z$ . Luego no será posible encontrar un estado  $\sigma'$  que efectúe el trabajo de  $\delta$  y de  $\sigma$  en las variables libres: ninguna variable al ejecutar  $c$  en  $\sigma'$  experimentará el cambio que sufre  $z$  al ejecutar  $c/\delta$ . O sea, aunque se satisfaga  $\sigma'x = \sigma z = \sigma'y$ , se tiene

$$\llbracket z := z - 1; z := 2 * z \rrbracket \sigma z \neq \llbracket x := x - 1; y := 2 * y \rrbracket \sigma'x$$

$$\llbracket z := z - 1; z := 2 * z \rrbracket \sigma z \neq \llbracket x := x - 1; y := 2 * y \rrbracket \sigma'y$$

El problema surge porque  $\delta$  no es inyectiva.

Ahora sí:

**Teorema de Sustitución.** Si  $\delta$  es inyectiva sobre  $FV(c)$  y  $\forall w \in FV(c). \sigma(\delta w) = \sigma'w$ , entonces

- o bien  $\llbracket c/\delta \rrbracket \sigma = \perp = \llbracket c \rrbracket \sigma'$ ,
- o bien  $\llbracket c/\delta \rrbracket \sigma \neq \perp \neq \llbracket c \rrbracket \sigma'$  y  $\forall w \in FV(c). \llbracket c/\delta \rrbracket \sigma(\delta w) = \llbracket c \rrbracket \sigma'w$ .

La demostración del TS requiere de una generalización:

**Lema de Sustitución (LS).** Sea  $FV(c) \subseteq V \subseteq \langle var \rangle$  tal que  $\delta$  es inyectiva sobre  $V$  y  $\forall w \in V. \sigma(\delta w) = \sigma'w$ . Entonces,

- o bien  $\llbracket c/\delta \rrbracket \sigma = \perp = \llbracket c \rrbracket \sigma'$ ,
- o bien  $\llbracket c/\delta \rrbracket \sigma \neq \perp \neq \llbracket c \rrbracket \sigma'$  y  $\forall w \in V. \llbracket c/\delta \rrbracket \sigma(\delta w) = \llbracket c \rrbracket \sigma'w$ .

#### RESUMEN: LENGUAJE IMPERATIVO SIMPLE (LIS)

Hemos presentado el lenguaje imperativo simple y dado las siguientes ecuaciones semánticas:

$$\begin{aligned} \llbracket \ ] &\in \langle intexp \rangle \rightarrow \Sigma \rightarrow \mathbb{Z} \\ \llbracket 0 \rrbracket \sigma &= 0 \\ \llbracket -e \rrbracket \sigma &= -\llbracket e \rrbracket \sigma \\ \llbracket e_0 + e_1 \rrbracket \sigma &= \llbracket e_0 \rrbracket \sigma + \llbracket e_1 \rrbracket \sigma && \text{similarmente para } -, *, \text{ etc.} \\ \llbracket v \rrbracket \sigma &= \sigma v \end{aligned}$$

$$\begin{aligned} \llbracket \ ] &\in \langle boolexp \rangle \rightarrow \Sigma \rightarrow \mathbb{B} \\ \llbracket \text{true} \rrbracket \sigma &= V \\ \llbracket \text{false} \rrbracket \sigma &= F \\ \llbracket e_0 = e_1 \rrbracket \sigma &= \llbracket e_0 \rrbracket \sigma = \llbracket e_1 \rrbracket \sigma && \text{similarmente para } <, >, \text{ etc.} \\ \llbracket \neg b \rrbracket \sigma &= \neg \llbracket b \rrbracket \sigma \\ \llbracket b_0 \wedge b_1 \rrbracket \sigma &= \llbracket b_0 \rrbracket \sigma \wedge \llbracket b_1 \rrbracket \sigma && \text{similarmente para } <, >, \vee, \text{ etc.} \end{aligned}$$

$$\begin{aligned} \llbracket \ ] &\in \langle comm \rangle \rightarrow \Sigma \rightarrow \Sigma_{\perp} \\ \llbracket \text{skip} \rrbracket \sigma &= \sigma \\ \llbracket v := e \rrbracket \sigma &= [\sigma|v : \llbracket e \rrbracket \sigma] \\ \llbracket \text{if } b \text{ then } c_0 \text{ else } c_1 \rrbracket \sigma &= \begin{cases} \llbracket c_0 \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c_1 \rrbracket \sigma & \text{si no} \end{cases} \\ \llbracket c_0; c_1 \rrbracket \sigma &= \llbracket c_1 \rrbracket_{\perp}(\llbracket c_0 \rrbracket \sigma) \\ \llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma &= (\lambda \sigma' \in \Sigma. [\sigma'|v : \sigma v])_{\perp}(\llbracket c \rrbracket [\sigma|v : \llbracket e \rrbracket \sigma]) \\ \llbracket \text{while } b \text{ do } c \rrbracket &= \bigsqcup_{i=0}^{\infty} F^i \perp_{\Sigma \rightarrow \Sigma_{\perp}} \end{aligned}$$

donde

$$\begin{aligned} f_{\perp} x &= \begin{cases} \perp & \text{si } x = \perp \\ f x & \text{si no} \end{cases} \\ F w \sigma &= \begin{cases} w_{\perp}(\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \\ \sigma & \text{si no} \end{cases} \end{aligned}$$

*Observación.* La ecuación del **while** es expresada normalmente sin explicitar el estado  $\sigma$ , pero también puede explicitarse

$$\llbracket \text{while } b \text{ do } c \rrbracket \sigma = \bigsqcup_{i=0}^{\infty} F^i \perp_{\Sigma \rightarrow \Sigma_{\perp}} \sigma$$

donde ahora el supremo es el del dominio llano  $\Sigma_{\perp}$ , a diferencia del supremo de la definición anterior que es el del dominio  $\Sigma \rightarrow \Sigma_{\perp}$ . La equivalencia entre ambas definiciones es consecuencia de la definición de supremo de una cadena de funciones, que es el supremo punto a punto:

$$\left( \bigsqcup_{i=0}^{\infty} \Sigma \rightarrow \Sigma_{\perp} F^i \perp_{\Sigma \rightarrow \Sigma_{\perp}} \right) \sigma = \bigsqcup_{i=0}^{\infty} \Sigma_{\perp} F^i \perp_{\Sigma \rightarrow \Sigma_{\perp}} \sigma$$

## 3. FALLAS EN EL LENGUAJE IMPERATIVO

Para la incorporación de la posibilidad de transferencia de control por fallas, se agregan excepciones al lenguaje:

$$\langle comm \rangle ::= \mathbf{fail} \mid \mathbf{catchin} \langle comm \rangle \mathbf{with} \langle comm \rangle$$

Ahora un programa tiene 3 comportamientos posibles:

- (1) da un estado final
- (2) aborta y da un estado final
- (3) no termina

Sea  $\Sigma' = \Sigma \cup \{\mathbf{abort}\} \times \Sigma$  con el orden discreto. La función semántica tendrá ahora tipo:

$$\llbracket \_ \rrbracket \in \langle comm \rangle \rightarrow \Sigma \rightarrow \Sigma'_\perp$$

En  $\Sigma'_\perp$  seguimos teniendo un dominio llano. Las ecuaciones semánticas son:

$$\begin{aligned} \llbracket \mathbf{skip} \rrbracket \sigma &= \sigma \\ \llbracket \mathbf{fail} \rrbracket \sigma &= \langle \mathbf{abort}, \sigma \rangle \\ \llbracket v := e \rrbracket \sigma &= [\sigma | v : \llbracket e \rrbracket \sigma] \\ \llbracket \mathbf{if} \ b \ \mathbf{then} \ c_0 \ \mathbf{else} \ c_1 \rrbracket \sigma &= \begin{cases} \llbracket c_0 \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c_1 \rrbracket \sigma & \text{si no} \end{cases} \end{aligned}$$

Para establecer las ecuaciones semánticas de las restantes construcciones, es necesario disponer de tres operadores de transferencia de control similar al operador  $(\_)_{\perp}$ . Pasamos ahora a definirlos.

Dada  $f \in \Sigma \rightarrow \Sigma'_\perp$ , denotamos por  $f_*$  la siguiente extensión de  $f$  a  $\Sigma'_\perp$ :

$$f_* \in \Sigma'_\perp \rightarrow \Sigma'_\perp$$

$$f_* x = \begin{cases} f \sigma & \text{si } x = \sigma \in \Sigma \\ x & \text{si no} \end{cases}$$

En este caso, la presencia de una situación abortiva determina que no se transfiere el control a  $f$ . Servirá para describir el significado de  $c_0; c_1$ , ya que si ocurre una situación de excepción al ejecutar  $c_0$ , el control no es transferido a  $c_1$ .

Por otro lado, dado  $f \in \Sigma \rightarrow \Sigma'_\perp$ , denotamos por  $f_+$  la siguiente extensión de  $f$  a  $\Sigma'_\perp$ :

$$f_+ \in \Sigma'_\perp \rightarrow \Sigma'_\perp$$

$$f_+ x = \begin{cases} f \sigma & \text{si } x = \langle \mathbf{abort}, \sigma \rangle \in \{\mathbf{abort}\} \times \Sigma \\ x & \text{si no} \end{cases}$$

En una clara dualidad con la definición de  $f_*$ , la definición de  $f_+$  determina lo contrario: se transfiere el control a  $f$  sólo en caso de excepción. Esto corresponderá a  $\mathbf{catchin} \ c \ \mathbf{with} \ c'$ .

Finalmente, dado  $f \in \Sigma \rightarrow \Sigma$ , denotamos por  $f_{\dagger}$  la siguiente extensión de  $f$  a  $\Sigma'_\perp$ :

$$f_{\dagger} \in \Sigma'_\perp \rightarrow \Sigma'_\perp$$

$$f_{\dagger} x = \begin{cases} \langle \mathbf{abort}, f \sigma \rangle & x = \langle \mathbf{abort}, \sigma \rangle \\ f x & x \in \Sigma \\ \perp & x = \perp \end{cases}$$

Note que aquí hay una transferencia de control a  $f$  en cualquier situación (abortiva o no). Servirá para restaurar el valor de las variables locales.

El sentido completo de estas funciones quedará claro al analizar las ecuaciones semánticas que siguen.

$$\begin{aligned} \llbracket c_0; c_1 \rrbracket \sigma &= \llbracket c_1 \rrbracket_* (\llbracket c_0 \rrbracket \sigma) \\ \llbracket \mathbf{catchin} \ c_0 \ \mathbf{with} \ c_1 \rrbracket \sigma &= \llbracket c_1 \rrbracket_+ (\llbracket c_0 \rrbracket \sigma) \\ \llbracket \mathbf{newvar} \ v := e \ \mathbf{in} \ c \rrbracket \sigma &= (\lambda \sigma' \in \Sigma. [\sigma' | v : \sigma \ v])_{\dagger} (\llbracket c \rrbracket [\sigma | v : \llbracket e \rrbracket \sigma]) \\ \llbracket \mathbf{while} \ b \ \mathbf{do} \ c \rrbracket &= \bigsqcup_{i=0}^{\infty} F^i \perp_{\Sigma \rightarrow \Sigma_\perp} \end{aligned}$$

donde

$$F \ w \ \sigma = \begin{cases} w_* (\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \\ \sigma & \text{si no} \end{cases}$$

**Ejercicio.** Reformular el teorema de coincidencia y el de sustitución para que tenga en cuenta los posibles nuevos comportamientos.

## 4. OUTPUT

Agregamos el comando

$$\langle comm \rangle ::= ! \langle \mathit{interp} \rangle$$

donde  $!e$  es el comando que escribe el valor de  $e$ . Claramente este comando puede encontrarse dentro de un ciclo o en cualquier fragmento de programa, eventualmente un programa puede generar una cantidad finita o infinita de output.

Los comportamientos posibles de un programa en un estado dado son ahora los siguientes:

- se genera una cantidad finita de output y luego "se cuelga"
- se genera una cantidad finita de output y luego termina
- se genera una cantidad finita de output y luego falla
- se genera una cantidad infinita de output

Sea  $\Omega$  = conjunto de estos comportamientos. El mismo será la unión de las siguientes familias. Las mismas corresponden en orden a las situaciones de arriba.

- $\{\langle n_1, \dots, n_k \rangle : n_i \in \mathbf{Z}\}$
- $\{\langle n_1, \dots, n_k, \sigma \rangle : n_i \in \mathbf{Z}, \sigma \in \Sigma\}$
- $\{\langle n_1, \dots, n_k, \langle \mathbf{abort}, \sigma \rangle \rangle : n_i \in \mathbf{Z}, \sigma \in \Sigma\}$
- $\{\langle n_1, \dots, n_k, \dots \rangle : n_i \in \mathbf{Z}\}$

En  $\Omega$  se define la relación  $\omega \leq \omega'$  cuando  $\omega$  es segmento inicial de  $\omega'$ . Con esta relación  $\Omega$  es un dominio, donde el mínimo es la secuencia vacía  $\langle \rangle$ . Las cadenas interesantes tienen supremo de la forma  $\langle n_1, \dots, n_k, \dots \rangle$ .

Las ecuaciones que definen la función semántica

$$\llbracket \_ \rrbracket \in \langle comm \rangle \rightarrow \Sigma \rightarrow \Omega$$

son las siguientes. La primera define la nueva construcción:

$$\llbracket !e \rrbracket \sigma = \langle \llbracket e \rrbracket \sigma, \sigma \rangle$$

Las restantes se mantienen invariantes respecto del lenguaje sin output, sólo que se deben redefinir los operadores de transferencia de control.

$$\begin{aligned} \llbracket \text{skip} \rrbracket \sigma &= \langle \sigma \rangle \\ \llbracket \text{fail} \rrbracket \sigma &= \langle \langle \text{abort}, \sigma \rangle \rangle \\ \llbracket v := e \rrbracket \sigma &= \langle \langle \sigma | v : \llbracket e \rrbracket \sigma \rangle \rangle \\ \llbracket \text{if } b \text{ then } c_0 \text{ else } c_1 \rrbracket \sigma &= \begin{cases} \llbracket c_0 \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c_1 \rrbracket \sigma & \text{si no} \end{cases} \\ \llbracket c_0; c_1 \rrbracket \sigma &= \llbracket c_1 \rrbracket_* (\llbracket c_0 \rrbracket \sigma) \\ \llbracket \text{catchin } c_0 \text{ with } c_1 \rrbracket \sigma &= \llbracket c_1 \rrbracket_+ (\llbracket c_0 \rrbracket \sigma) \\ \llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma &= \langle \lambda \sigma' \in \Sigma. [\sigma' | v : \sigma \ v] \rangle_! (\llbracket c \rrbracket [\sigma | v : \llbracket e \rrbracket \sigma]) \\ \llbracket \text{while } b \text{ do } c \rrbracket &= \bigsqcup_{i=0}^{\infty} F^i \perp_{\Sigma \rightarrow \Omega} \end{aligned}$$

donde

$$F \ w \ \sigma = \begin{cases} w_* (\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \\ \langle \sigma \rangle & \text{si no} \end{cases}$$

Los nuevos operadores de transferencia de control deben considerar los distintos tipos de comportamiento esperado. Sea  $f \in \Sigma \rightarrow \Omega$ , entonces las extensiones

$$f_*, f_+ \in \Omega \rightarrow \Omega$$

se definen de la siguiente manera. El operador  $(\_)*$  transfiere el control a la función sólo en caso de terminación normal:

$$f_* x = \begin{cases} \langle n_1, \dots, n_k \rangle & x = \langle n_1, \dots, n_k \rangle \\ \langle n_1, \dots, n_k, f\sigma \rangle & x = \langle n_1, \dots, n_k, \sigma \rangle \\ \langle n_1, \dots, n_k, \langle \text{abort}, \sigma \rangle \rangle & x = \langle n_1, \dots, n_k, \langle \text{abort}, \sigma \rangle \rangle \\ \langle n_1, \dots, n_k, \dots \rangle & x = \langle n_1, \dots, n_k, \dots \rangle \end{cases}$$

Note que una forma de definir  $(\_)*$  de manera más compacta es:

$$f_* x = \begin{cases} \langle \rangle & x = \langle \rangle \\ f\sigma & x = \langle \sigma \rangle \\ \langle \text{abort}, \sigma \rangle & x = \langle \text{abort}, \sigma \rangle \\ \langle n \rangle ++ f_* \omega & x = \langle n \rangle ++ \omega \end{cases}$$

De la misma manera definimos:

$$f_+ x = \begin{cases} \langle \rangle & x = \langle \rangle \\ \langle \sigma \rangle & x = \langle \sigma \rangle \\ f\sigma & x = \langle \text{abort}, \sigma \rangle \\ \langle n \rangle ++ f_+ \omega & x = \langle n \rangle ++ \omega \end{cases}$$

Sea  $f \in \Sigma \rightarrow \Omega$ , entonces la extensión

$$f_! \in \Omega \rightarrow \Omega$$

se define

$$f_! x = \begin{cases} \langle \rangle & x = \langle \rangle \\ \langle f\sigma \rangle & x = \langle \sigma \rangle \\ \langle \text{abort}, f\sigma \rangle & x = \langle \text{abort}, \sigma \rangle \\ \langle n \rangle ++ f_! \omega & x = \langle n \rangle ++ \omega \end{cases}$$

En las siguientes secciones abordaremos herramientas que nos permitirán definir dominios de manera apropiada, atendiendo a la complejidad que los mismos comienzan a evidenciar cuando avanzamos sobre las extensiones de LIS.

## 5. MÁS SOBRE DOMINIOS

### Producto Cartesiano.

*Producto de cpos.* Si  $P_0, P_1, \dots, P_{n-1}$  son órdenes parciales, entonces  $P_0 \times P_1, \dots \times P_{n-1}$  también lo es, donde el orden entre tuplas se define componente a componente.

*Cadenas.* Una cadena de tuplas es una secuencia

$$\begin{aligned} \langle p_0^1, p_1^1, \dots, p_{n-1}^1 \rangle &\leq \\ \langle p_0^2, p_1^2, \dots, p_{n-1}^2 \rangle &\leq \\ &\vdots \\ \langle p_0^n, p_1^n, \dots, p_{n-1}^n \rangle &\leq \\ &\vdots \end{aligned}$$

tal que si tomamos componente a componente formamos cadenas en los respectivos órdenes:

$$\begin{aligned} p_0^1 &\leq p_0^2 \leq \dots \leq p_0^n \leq \dots \\ p_1^1 &\leq p_1^2 \leq \dots \leq p_1^n \leq \dots \\ &\vdots \end{aligned}$$

*Producto de predomios.* Si  $P_0, P_1, \dots, P_{n-1}$  son predomios, entonces  $P_0 \times P_1, \dots \times P_{n-1}$  también lo es, donde el supremo de una cadena de tuplas se define componente a componente.

*Producto de dominios.* Si  $P_0, P_1, \dots, P_{n-1}$  son dominios, entonces  $P_0 \times P_1, \dots \times P_{n-1}$  también lo es, donde el mínimo es la tupla que consiste del mínimo de cada uno de los dominios.

Todas las funciones sencillas usuales (proyecciones, constructor de tuplas, etc) son trivialmente continuas.

**Uniones Disjuntas.** Dados conjuntos  $P_0, P_1, \dots, P_{n-1}$  se define la unión disjunta

$$P_0 + P_1 + \dots + P_{n-1} = \{\langle i, p \rangle : p \in P_i\}.$$

Se definen las inyecciones  $\iota_i \in P_i \rightarrow P_0 + P_1 + \dots + P_{n-1}$  mediante  $\iota_i p = \langle i, p \rangle$

*Uniones Disjuntas de cpos.* Dados órdenes parciales  $P_0, P_1, \dots, P_{n-1}$ , entonces  $P_0 + P_1 + \dots + P_{n-1}$  es un orden parcial, donde el orden "no mezcla" los órdenes de los diferentes conjuntos dados:

$$\langle i, p \rangle \leq \langle j, q \rangle \iff i = j \wedge p \leq_i q$$

*Cadenas.* Una cadena de  $P_0 + P_1 + \dots + P_{n-1}$  es una que proviene enteramente de uno de los  $P_i$ :

$$\langle i, p_1 \rangle \leq \langle i, p_2 \rangle \leq \dots \leq \langle i, p_n \rangle \leq \dots$$

donde los  $p_j$  son todos elementos de  $P_i$ .

*Unión de predominios.* Dados predominios  $P_0, P_1, \dots, P_{n-1}$ , entonces  $P_0 + P_1 + \dots + P_{n-1}$  es un predominio, donde el supremo de una cadena es simplemente el supremo de la componente de donde la cadena proviene:

$$\bigsqcup_{j=1}^{\infty} \langle i, p_j \rangle = \langle i, \bigsqcup_{j=1}^{\infty} p_j \rangle$$

*Unión de dominios.* Dados dominios  $P_0, P_1, \dots, P_{n-1}$ , entonces  $P_0 + P_1 + \dots + P_{n-1}$  en general **no** es un dominio (sólo lo es en el caso trivial  $n = 1$ ). ¿Cuál sería el mínimo, si los mínimos de los diferentes  $P_i$  no se comparan entre sí?

Todas las funciones sencillas usuales (inyecciones, análisis por casos, etc) son trivialmente continuas.

**Dominios recursivos.** El domino semántico para LIS con fallas y output puede ser definido a través de una **ecuación recursiva de dominos** que exprese el hecho de que un objeto  $\omega$  puede ser visto como el tramo final de otros objetos pertenecientes a  $\Omega$ : para todo  $k$  entero,  $k \dashv\vdash \omega$  es también un objeto de  $\Omega$ . Esto es:

$$\Omega \approx (\Sigma' + \mathbf{Z} \times \Omega)_{\perp}$$

El símbolo  $\approx$  significa isomorfismo, el cual está dado por las funciones continuas (una inversa de la otra):

$$\phi \in \Omega \rightarrow (\Sigma' + \mathbf{Z} \times \Omega)_{\perp} \quad \psi \in (\Sigma' + \mathbf{Z} \times \Omega)_{\perp} \rightarrow \Omega$$

definidas mediante:

$$\phi x = \begin{cases} \perp & x = \langle \rangle \\ \iota_{\perp}(\iota_0 y) & x = \langle y \rangle \text{ con } y \in \Sigma' \\ \iota_{\perp}(\iota_1 \langle n, \phi \omega \rangle) & x = \langle n \rangle \dashv\vdash \omega \end{cases}$$

$$\psi x = \begin{cases} \langle \rangle & x = \perp \\ \langle y \rangle & x = \iota_{\perp}(\iota_0 y) \text{ con } y \in \Sigma' \\ \langle n \rangle \dashv\vdash \psi \omega & x = \iota_{\perp}(\iota_1 \langle n, \omega \rangle) \end{cases}$$

*Notación.* Para expresar las ecuaciones semánticas en adelante serán útiles las siguientes composiciones.

$$\begin{aligned} \iota_{term} &= \psi \cdot \iota_{\perp} \cdot \iota_0 \cdot \iota_{norm} \in \Sigma \rightarrow \Omega \\ \iota_{abort} &= \psi \cdot \iota_{\perp} \cdot \iota_0 \cdot \iota_{abnorm} \in \Sigma \rightarrow \Omega \\ \iota_{out} &= \psi \cdot \iota_{\perp} \cdot \iota_1 \in \mathbf{Z} \times \Sigma \rightarrow \Omega \\ \perp_{\Omega} &= \psi(\perp) \in \Omega \end{aligned}$$

Las mismas pueden expresarse en el siguiente gráfico:

$$\begin{array}{ccccc} \Sigma & \xrightarrow{\iota_{norm}} & \Sigma' & \xrightarrow{\iota_0} & \\ \Sigma & \xrightarrow{\iota_{abnorm}} & \Sigma' + \mathbf{Z} \times \Omega & \xrightarrow{\iota_{\perp}} & (\Sigma' + \mathbf{Z} \times \Omega)_{\perp} \xrightarrow{\psi} \Omega \\ & & \mathbf{Z} \times \Omega & \xrightarrow{\iota_1} & \end{array}$$

Utilizando esta nueva definición matemática del domino semántico (dominio recursivo), podemos reformular las ecuaciones semánticas dadas con anterioridad de la siguiente manera.

$$\begin{aligned} \llbracket \text{skip} \rrbracket \sigma &= \iota_{term} \sigma \\ \llbracket \text{fail} \rrbracket \sigma &= \iota_{abort} \sigma \\ \llbracket v := e \rrbracket \sigma &= \iota_{term} [\sigma | v : \llbracket e \rrbracket \sigma] \\ \llbracket !e \rrbracket \sigma &= \iota_{out} (\llbracket e \rrbracket \sigma, \iota_{term} \sigma) \\ \llbracket \text{if } b \text{ then } c_0 \text{ else } c_1 \rrbracket \sigma &= \begin{cases} \llbracket c_0 \rrbracket \sigma & \text{si } \llbracket b \rrbracket \sigma \\ \llbracket c_1 \rrbracket \sigma & \text{si no} \end{cases} \\ \llbracket c_0; c_1 \rrbracket \sigma &= \llbracket c_1 \rrbracket_* (\llbracket c_0 \rrbracket \sigma) \\ \llbracket \text{catchin } c_0 \text{ with } c_1 \rrbracket \sigma &= \llbracket c_1 \rrbracket_+ (\llbracket c_0 \rrbracket \sigma) \\ \llbracket \text{newvar } v := e \text{ in } c \rrbracket \sigma &= (\lambda \sigma' \in \Sigma. [\sigma' | v : \sigma v])_{\dagger} (\llbracket c \rrbracket [\sigma | v : \llbracket e \rrbracket \sigma]) \\ \llbracket \text{while } b \text{ do } c \rrbracket &= \bigsqcup_{i=0}^{\infty} F^i \perp_{\Sigma \rightarrow \Omega} \end{aligned}$$

donde

$$F w \sigma = \begin{cases} w_* (\llbracket c \rrbracket \sigma) & \text{si } \llbracket b \rrbracket \sigma \\ \iota_{term} \sigma & \text{si no} \end{cases}$$

Los operadores de transferencia de control pueden también ser redefinidos.

$$f_*x = \begin{cases} \perp_\Omega & x = \perp_\Omega \\ f\sigma & x = \iota_{term}\sigma \\ \iota_{abort}\sigma & x = \iota_{abort}\sigma \\ \iota_{out}(n, f_*\omega) & x = \iota_{out}(n, \omega) \end{cases}$$

$$f_+x = \begin{cases} \perp_\Omega & x = \perp_\Omega \\ \iota_{term}\sigma & x = \iota_{term}\sigma \\ f\sigma & x = \iota_{abort}\sigma \\ \iota_{out}(n, f_+\omega) & x = \iota_{out}(n, \omega) \end{cases}$$

$$f_{\dagger}x = \begin{cases} \perp_\Omega & x = \perp_\Omega \\ \iota_{term}(f\sigma) & x = \iota_{term}\sigma \\ \iota_{abort}(f\sigma) & x = \iota_{abort}\sigma \\ \iota_{out}(n, f_{\dagger}\omega) & x = \iota_{out}(n, \omega) \end{cases}$$

## 6. INPUT

Para poder expresar la acción de alojar en una variable un entero dado como input, agregamos el comando:

$$\langle comm \rangle ::= ? \langle var \rangle$$

Para poder establecer el significado de esta construcción debemos enriquecer el dominio semántico. Lo hacemos planteando una ecuación recursiva, y asumiendo que posee solución. La misma contempla (a través de la noción de función) la posibilidad de que el comportamiento de un programa dependa de un entero dado (input).

$$\Omega \approx (\Sigma' + \mathbf{Z} \times \Omega + \mathbf{Z} \rightarrow \Omega)_\perp$$

Como antes, el símbolo  $\approx$  significa isomorfismo, el cual está dado por las funciones continuas (una inversa de la otra):

$$\phi \in \Omega \rightarrow (\Sigma' + \mathbf{Z} \times \Omega + \mathbf{Z} \rightarrow \Omega)_\perp \quad \psi \in (\Sigma' + \mathbf{Z} \times \Omega + \mathbf{Z} \rightarrow \Omega)_\perp \rightarrow \Omega$$

*Notación.* Para expresar las ecuaciones semánticas, actualizamos la lista de composiciones:

$$\begin{aligned} \iota_{term} &= \psi \cdot \iota_\perp \cdot \iota_0 \cdot \iota_{norm} \in \Sigma \rightarrow \Omega \\ \iota_{abort} &= \psi \cdot \iota_\perp \cdot \iota_0 \cdot \iota_{abnorm} \in \Sigma \rightarrow \Omega \\ \iota_{out} &= \psi \cdot \iota_\perp \cdot \iota_1 \in \mathbf{Z} \times \Sigma \rightarrow \Omega \\ \iota_{in} &= \psi \cdot \iota_\perp \cdot \iota_2 \in (\mathbf{Z} \rightarrow \Sigma) \rightarrow \Omega \\ \perp_\Omega &= \psi(\perp) \in \Omega \end{aligned}$$

Las mismas pueden expresarse en un gráfico similar al dado para output:

$$\begin{array}{ccccc} \Sigma & \xrightarrow{\iota_{norm}} & \Sigma' & \xrightarrow{\iota_0} & \\ \Sigma & \xrightarrow{\iota_{abnorm}} & & & \\ & & \mathbf{Z} \times \Omega & \xrightarrow{\iota_1} & \Sigma' + \mathbf{Z} \times \Omega \xrightarrow{\iota_\perp} (\Sigma' + \mathbf{Z} \times \Omega)_\perp \xrightarrow{\psi} \Omega \\ & & \mathbf{Z} \rightarrow \Omega & \xrightarrow{\iota_2} & \end{array}$$

Dar las ecuaciones semánticas para el lenguaje completo requiere sólo agregar la ecuación del constructor  $?v$ , y actualizar las definiciones de los operadores de transferencia de control.

$$\llbracket ?v \rrbracket \sigma = \iota_{in}(\lambda n \in \mathbf{Z}. \iota_{term}[\sigma|v : n])$$

$$f_*x = \begin{cases} \perp_\Omega & x = \perp_\Omega \\ f\sigma & x = \iota_{term}\sigma \\ \iota_{abort}\sigma & x = \iota_{abort}\sigma \\ \iota_{out}(n, f_*\omega) & x = \iota_{out}(n, \omega) \\ \iota_{in}(f_* \cdot g) & x = \iota_{in}g \end{cases}$$

$$f_+x = \begin{cases} \perp_\Omega & x = \perp_\Omega \\ \iota_{term}\sigma & x = \iota_{term}\sigma \\ f\sigma & x = \iota_{abort}\sigma \\ \iota_{out}(n, f_+\omega) & x = \iota_{out}(n, \omega) \\ \iota_{in}(f_+ \cdot g) & x = \iota_{in}g \end{cases}$$

$$f_{\dagger}x = \begin{cases} \perp_\Omega & x = \perp_\Omega \\ \iota_{term}(f\sigma) & x = \iota_{term}\sigma \\ \iota_{abort}(f\sigma) & x = \iota_{abort}\sigma \\ \iota_{out}(n, f_{\dagger}\omega) & x = \iota_{out}(n, \omega) \\ \iota_{in}(f_{\dagger} \cdot g) & x = \iota_{in}g \end{cases}$$

## 7. SEMÁNTICA OPERACIONAL

Cuando hablamos de semántica operacional (o de transiciones), en vez de asociar significado a los programas a través de una función de los programas en los significados, se describe cómo se realiza el cómputo. Por ejemplo:

$$\langle x := 1; y := 2 * y, \sigma \rangle \rightarrow \langle y := 2 * y, [\sigma|x : 1] \rangle \rightarrow [\sigma|x : 1|y : 2 * \sigma y]$$

La relación  $\rightarrow$  describe un paso de ejecución. Por eso se la llama *small-step semantics*. En cada paso se pasa de una *configuración* a otra. El conjunto de configuraciones que describe las etapas de cómputo será denotado mediante  $\Gamma$ . El mismo está formado por configuraciones *terminales* (que reflejan el resultado de un cómputo), y por configuraciones *no terminales* (que reflejan una etapa intermedia del mismo).

$$\Gamma = \Gamma_t \cup \Gamma_n$$

Cada uno de estos conjuntos está determinado por:

$$\begin{aligned} \Gamma_t &= \Sigma \\ \Gamma_n &= \langle comm \rangle \times \Sigma \end{aligned}$$

El paso de ejecución  $\rightarrow$ , que lleva una configuración no terminal en una configuración (terminal o no), se define matemáticamente como una relación:

$$\rightarrow \subseteq \Gamma_n \times \Gamma$$

La definición de una relación de este tipo (formada por objetos que proceden de una sintaxis abstracta) suele hacerse de manera axiomática. Vamos a mostrar ahora los axiomas que definen la misma.

Los primeros obedecen a una clara intuición de cómputo, y representan casos de obtención de configuraciones terminales:

$$\overline{\langle \text{skip}, \sigma \rangle \rightarrow \sigma}$$

$$\overline{\langle v := e, \sigma \rangle \rightarrow [\sigma|v : \llbracket e \rrbracket \sigma]}$$

La posibilidad de obtener una secuencia encadenada de transiciones se representa axiomáticamente mediante:

$$\frac{\langle c_0, \sigma \rangle \rightarrow \sigma'}{\langle c_0; c_1, \sigma \rangle \rightarrow \langle c_1, \sigma' \rangle}$$

$$\frac{\langle c_0, \sigma \rangle \rightarrow \langle c'_0, \sigma' \rangle}{\langle c_0; c_1, \sigma \rangle \rightarrow \langle c'_0; c_1, \sigma' \rangle}$$

Note como estas reglas permiten construir un paso intermedio de ejecución, como por ejemplo el primer paso de la ejecución de arriba:

$$\frac{\langle x := 1, \sigma \rangle \rightarrow [\sigma|x : 1]}{\langle x := 1; y := 2 * y, \sigma \rangle \rightarrow \langle y := 2 * y, [\sigma|x : 1] \rangle}$$

Las reglas para el comando if y el comando while también obedecen a su intuición operacional:

$$\frac{(\llbracket e \rrbracket \sigma = V)}{\langle \text{if } e \text{ then } c \text{ else } c', \sigma \rangle \rightarrow \langle c, \sigma \rangle}$$

$$\frac{(\llbracket e \rrbracket \sigma = F)}{\langle \text{if } e \text{ then } c \text{ else } c', \sigma \rangle \rightarrow \langle c', \sigma \rangle}$$

$$\frac{(\llbracket e \rrbracket \sigma = F)}{\langle \text{while } e \text{ do } c, \sigma \rangle \rightarrow \sigma}$$

$$\frac{(\llbracket e \rrbracket \sigma = T)}{\langle \text{while } e \text{ do } c, \sigma \rangle \rightarrow \langle c : \text{ while } e \text{ do } c, \sigma \rangle}$$

Finalmente debemos dar reglas para el comando newvar. Una posible regla que podría considerarse natural es:

$$\overline{\langle \text{newvar } v := e \text{ in } c, \sigma \rangle \rightarrow \langle c; v := \sigma v, [\sigma|v : \llbracket e \rrbracket \sigma] \rangle}$$

La dificultad de esta regla es la pérdida de la localia de  $v$  en  $c$ , que tendría inconvenientes en una eventual extensión del lenguaje con concurrencia. En consistencia con el Reynolds vamos a adoptar:

$$\frac{\langle c, [\sigma|v : \llbracket e \rrbracket \sigma] \rangle \rightarrow \sigma'}{\langle \text{newvar } v := e \text{ in } c, \sigma \rangle \rightarrow [\sigma'|v : \sigma v]}$$

$$\frac{\langle c, [\sigma|v : \llbracket e \rrbracket \sigma] \rangle \rightarrow \langle c', \sigma' \rangle}{\langle \text{newvar } v := e \text{ in } c, \sigma \rangle \rightarrow \langle \text{newvar } v := \sigma' v \text{ in } c', [\sigma'|v : \sigma v] \rangle}$$



## 8. CORRECCIÓN DE LA SEMÁNTICA OPERACIONAL

Note que la relación  $\rightarrow$  así definida es una función: en efecto, ninguna configuración no terminal puede mover (en un sólo paso) hacia más de una configuración. Esto significa que la relación es determinística. Además ninguna configuración no terminal puede mover hacia menos de una configuración (no se traba).

En efecto, sea  $\langle c, \sigma \rangle$  una configuración no terminal. Entonces, si

- si  $c = \mathbf{skip}$ , hay una única transición posible, a  $\sigma$ .
- si  $c = (v := e)$ , hay una única transición posible, a  $[\sigma|v : \llbracket e \rrbracket \sigma]$ .
- lo mismo se puede establecer para el **if** y el **while**, dependiendo de el valor de verdad de la guarda.
- si  $c = c_0; c_1$ , entonces, por hipótesis inductiva desde  $\langle c_0, \sigma \rangle$  hay una única transición posible. Si esa transición es a una configuración de la forma  $\sigma'$ , entonces hay una única transición posible desde  $\langle c, \sigma \rangle$ , es a  $\langle c_1, \sigma' \rangle$ . Si en cambio esa transición es a una configuración de la forma  $\langle c'_0, \sigma \rangle$ , entonces hay una única transición posible desde  $\langle c, \sigma \rangle$ , es a  $\langle c'_0; c_1, \sigma' \rangle$ .

Como  $\rightarrow$  es una función, para toda configuración existe una única ejecución. Por *ejecución* entendemos una secuencia  $c_0 \rightarrow c_1 \rightarrow c_2 \rightarrow \dots$  maximal, esto es, que no puede prolongarse más de lo que está. Dicha ejecución es infinita o termina en una configuración terminal  $\sigma$ . Si la ejecución es infinita decimos que  $c_0$  *diverge* y escribimos  $c_0 \uparrow$ . Se puede entonces definir:

$$\llbracket c \rrbracket \sigma = \begin{cases} \perp & \text{si } \langle c, \sigma \rangle \uparrow \\ \sigma' & \text{si existe } \sigma' \text{ tal que } \langle c, \sigma \rangle \rightarrow^* \sigma' \end{cases}$$

donde  $\rightarrow^*$  es la clausura reflexiva y transitiva definida a continuación.

$$\frac{\gamma \rightarrow \gamma'}{\gamma \rightarrow^* \gamma'}$$

$$\overline{\gamma \rightarrow^* \gamma}$$

$$\frac{\gamma \rightarrow^* \gamma' \quad \gamma' \rightarrow^* \gamma''}{\gamma \rightarrow^* \gamma''}$$

Se puede demostrar que  $\llbracket c \rrbracket = \llbracket c \rrbracket$ . Damos ahora una serie de resultados que están involucrados en la prueba.

**Lema 1.**

- (1) Si  $\langle c_0, \sigma \rangle \rightarrow^* \sigma'$ , entonces  $\langle c_0; c_1, \sigma \rangle \rightarrow^* \langle c_1, \sigma' \rangle$
- (2) Si  $\langle c, [\sigma|v : \llbracket e \rrbracket \sigma] \rangle \rightarrow^* \sigma'$ , entonces

$$\langle \mathbf{newvar} \ v := e \ \mathbf{in} \ c, \sigma \rangle \rightarrow^* [\sigma'|v : \sigma v].$$

- (3) Si  $\langle c, [\sigma|v : \llbracket e \rrbracket \sigma] \rangle \rightarrow^* \langle c', \sigma' \rangle$ , entonces

$$\langle \mathbf{newvar} \ v := e \ \mathbf{in} \ c, \sigma \rangle \rightarrow^* \langle \mathbf{newvar} \ v := \sigma' x \ \mathbf{in} \ c', [\sigma'|v : \sigma v] \rangle$$

**Lema 2.**

- (1)  $\langle c, \sigma \rangle \rightarrow \sigma' \implies \llbracket c \rrbracket \sigma = \sigma'$ .
- (2)  $\langle c, \sigma \rangle \rightarrow \langle c', \sigma' \rangle \implies \llbracket c \rrbracket \sigma = \llbracket c' \rrbracket \sigma'$ .

**Lema 3.**  $\llbracket c \rrbracket \sigma = \sigma' \implies \langle c, \sigma \rangle \rightarrow^* \sigma'$ 

**Teorema.** Para todo comando  $c$  se tiene  $\llbracket c \rrbracket = \llbracket c \rrbracket$ .

**8.1. Semántica operacional de las fallas.** Para dar la semántica de transiciones de LIS con fallas, modificamos el conjunto de configuraciones terminales:

$$\Gamma_t = \Sigma \cup \{\mathbf{abort}\} \times \Sigma$$

Corresponde además agregar reglas que involucren las nuevas configuraciones terminales:

$$\overline{\langle \mathbf{fail}, \sigma \rangle \rightarrow \langle \mathbf{abort}, \sigma \rangle}$$

$$\frac{\langle c_0, \sigma \rangle \rightarrow \langle \mathbf{abort}, \sigma' \rangle}{\langle c_0; c_1, \sigma \rangle \rightarrow \langle \mathbf{abort}, \sigma' \rangle}$$

El **catchin** se define por tres reglas

$$\frac{\langle c_0, \sigma \rangle \rightarrow \sigma'}{\langle \mathbf{catchin} \ c_0 \ \mathbf{with} \ c_1, \sigma \rangle \rightarrow \sigma'}$$

$$\frac{\langle c_0, \sigma \rangle \rightarrow \langle \mathbf{abort}, \sigma' \rangle}{\langle \mathbf{catchin} \ c_0 \ \mathbf{with} \ c_1, \sigma \rangle \rightarrow \langle c_1, \sigma' \rangle}$$

$$\frac{\langle c_0, \sigma \rangle \rightarrow \langle c'_0, \sigma' \rangle}{\langle \mathbf{catchin} \ c_0 \ \mathbf{with} \ c_1, \sigma \rangle \rightarrow \langle \mathbf{catchin} \ c'_0 \ \mathbf{with} \ c_1, \sigma' \rangle}$$

Por último, al **newvar** se le agrega un caso más

$$\frac{\langle c, \sigma \rangle \rightarrow \langle \mathbf{abort}, \sigma' \rangle}{\langle \mathbf{newvar} \ v := e \ \mathbf{in} \ c, \sigma \rangle \rightarrow \langle \mathbf{abort}, [\sigma'|v : \sigma v] \rangle}$$

El **if** y el **while** habían sido definidos con suficiente generalidad para que no requieran revisión.

La relación  $\rightarrow$  sigue siendo una función en el lenguaje con fallas, toda configuración  $\gamma$  tiene una única ejecución que puede ser infinita ( $\gamma$  diverge) o terminar en una configuración terminal que puede ser de la forma  $\sigma$  o  $\langle \mathbf{abort}, \sigma \rangle$ .

Se puede definir:

$$\llbracket c \rrbracket \sigma = \begin{cases} \perp & \text{si } \langle c, \sigma \rangle \uparrow \\ \sigma' & \text{si existe } \sigma' \text{ tal que } \langle c, \sigma \rangle \rightarrow^* \sigma' \\ \langle \mathbf{abort}, \sigma' \rangle & \text{si existe } \sigma' \text{ tal que } \langle c, \sigma \rangle \rightarrow^* \langle \mathbf{abort}, \sigma' \rangle \end{cases}$$

y obtendremos de manera similar a LIS que para todo comando  $c$  se tiene  $\llbracket c \rrbracket = \llbracket c \rrbracket$ .